UCS503- Project Report

UCS 503 – SOFTWARE ENGINEERING

SERVECONNECT

A Hyper-Local Service Professional Marketplace

Submitted by:

Vijay Kiran (Backend & Database)
Pratham Goyal (Team Lead, Admin, Integration & Testing)
Keshav Goyal (Frontend UI/UX)

Submitted to:

Mr.Hardik Gujral

Mid-Semester Evaluation

Computer Science and Engineering Department

Contents

1 Introduction **2**
1.1 Background and Context 2
1.1.1 Problem Statement 2
1.2 Project Objectives 2

2 Methodology and Design **3**
2.1 System Architecture 3
2.1.1 High-Level Design 3
2.1.2 Technical Stack 3
2.2 Implementation Details 3
2.2.1 Module 1: Core Functionality (Auth & Admin) 3
2.2.2 Module 2: Secondary Features (Search & Bookings) 3
2.2.3 System Overview 4

3 Results and Evaluation **9**
3.1 Testing Methodology 9
3.2 Analysis 9

4 Conclusion **10**
4.1 Summary of Work 10
4.2 Key Findings 10
4.3 Future Work and Recommendations 10
4.4 Final Remarks 10

1 Introduction

1.1 Background and Context

The hyper-local service industry currently struggles with fragmented communication, unverified workers, and inefficient scheduling. ServeConnect addresses this gap by providing a unified, transparent platform where customers can easily find, verify, and book local service professionals (such as electricians, plumbers, and carpenters). By integrating automated administrative verification and an intelligent booking system, the platform ensures trust and reliability.

1.1.1 Problem Statement

Customers face inefficiencies and safety concerns when trying to find reliable daily-wage workers and service professionals. There is a need for a comprehensive platform that not only allows workers to register their services but also ensures they are strictly vetted by an admin before becoming publicly visible. Furthermore, the system needs a structured booking flow to prevent schedule overlaps (double-booking) and facilitate seamless communication.

1.2 Project Objectives

1. **Objective 1:** Develop a robust core platform with role-based JWT authentication allowing Customers, Workers, and Admins to register and securely log in.
2. **Objective 2:** Implement an Admin Verification Module that keeps new workers hidden from public search until verified, triggering an automated Twilio SMS notification to the worker upon approval.
3. **Objective 3:** Build a dynamic, location-aware Customer Search engine (Zomato-style global location context) with category filtering and automated rating-based ranking logic.
4. **Objective 4:** Develop a Scheduling & Booking system that allows customers to request time slots while enforcing backend constraints to prevent double-booking of workers.

2 Methodology and Design

2.1 System Architecture

The ServeConnect architecture is built around three primary actors: Customers, Workers, and Admins. The workflow is designed to handle user verification, service search, and secure appointment scheduling efficiently, as modeled in the diagrams below.

2.1.1 High-Level Design

The system utilizes a modular MERN-stack architecture. When a worker submits a registration via the frontend, the data is securely hashed and stored in MongoDB with a "pending" status. The Admin Dashboard interacts with the backend to approve the worker, triggering the `smsService` (Twilio API). Customers navigate a state-managed React frontend (using Context API for global location). When a customer books a worker, the backend intercepts the request to query existing database records, ensuring the requested time slot does not overlap with an already "accepted" or "pending" booking.

2.1.2 Technical Stack

- **Frontend / UI:** React.js, Vite, React Router DOM, Custom CSS (Modern UI/UX).
- **Backend / API:** Node.js, Express.js, RESTful API architecture.
- **Database:** MongoDB (NoSQL) with Mongoose ORM.
- **Testing:** Jest and Supertest for automated Integration Testing.
- **External APIs:** Twilio SDK (SMS Notifications), OpenStreetMap/Nominatim API (Reverse Geocoding for automatic location detection).

2.2 Implementation Details

2.2.1 Module 1: Core Functionality (Auth & Admin)

The core functionality handles user authentication, registration, and login flows via `authController` and `authMiddleware`. The `adminController` handles the verification lifecycle of workers, ensuring database integrity and triggering external SMS notifications upon status changes.

2.2.2 Module 2: Secondary Features (Search & Bookings)

The `workerController` serves as the search layer, utilizing regex-based location and category matching, coupled with MongoDB sorting logic. The `bookingController` manages the transactional scheduling between Customers and Workers, utilizing stateful dashboard views on the frontend for Workers to Accept/Reject incoming jobs.

2.2.3 System Overview

Activity Diagram

Customer – Booking Flow

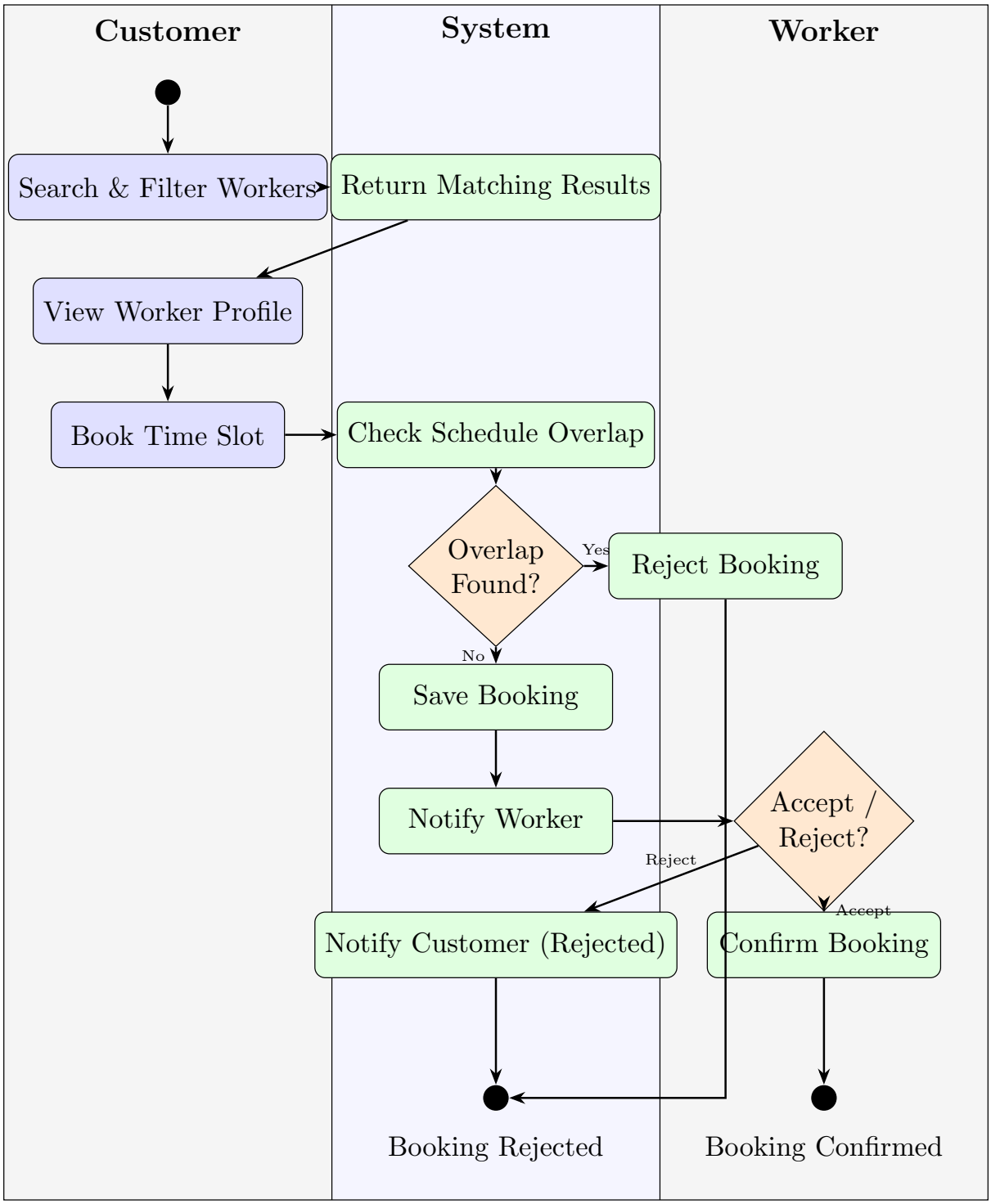


Figure 2.1: Activity Diagram – Customer Booking Flow

Admin – Worker Verification Flow

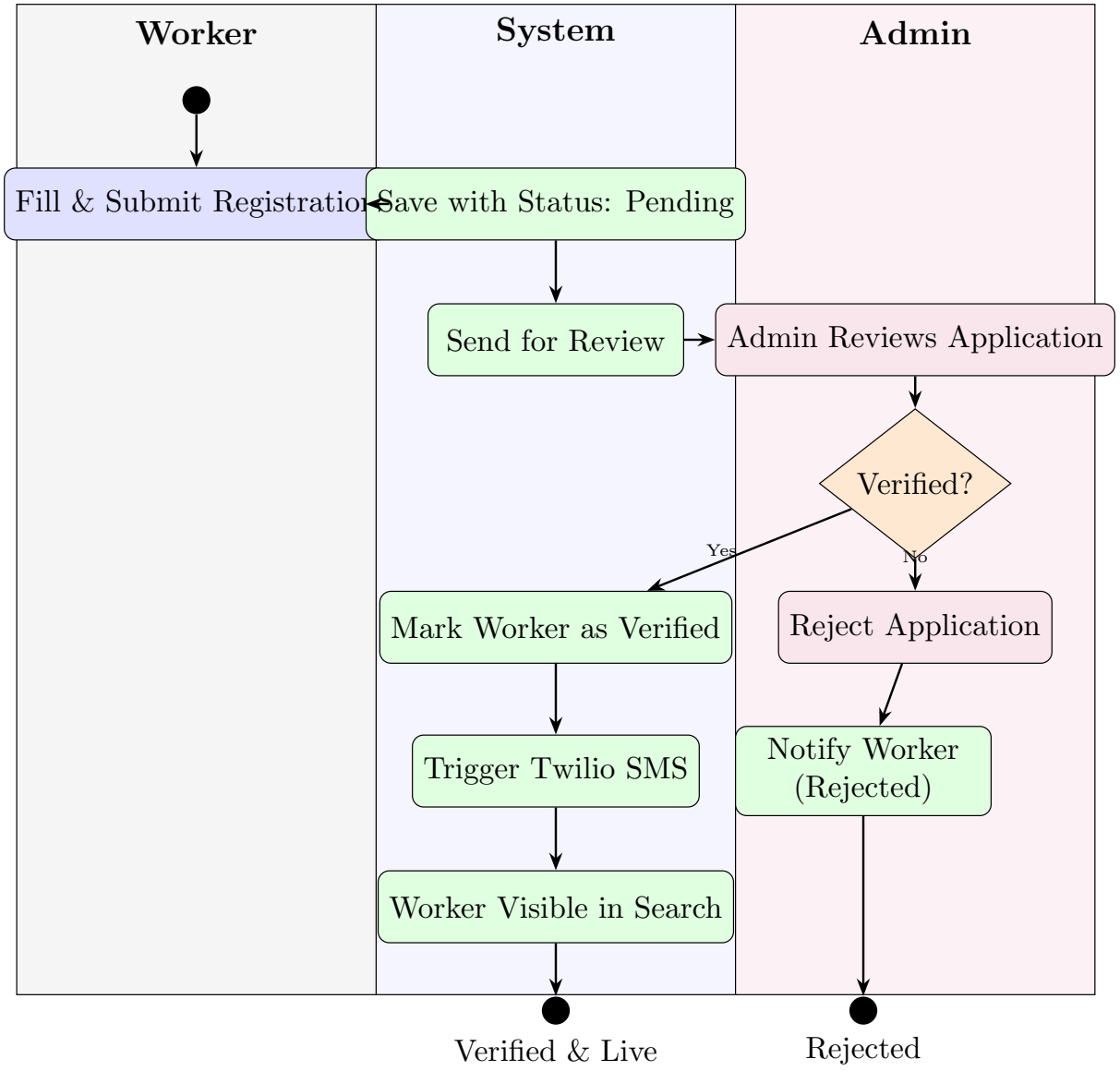


Figure 2.2: Activity Diagram – Admin Worker Verification Flow

Use Case Diagram

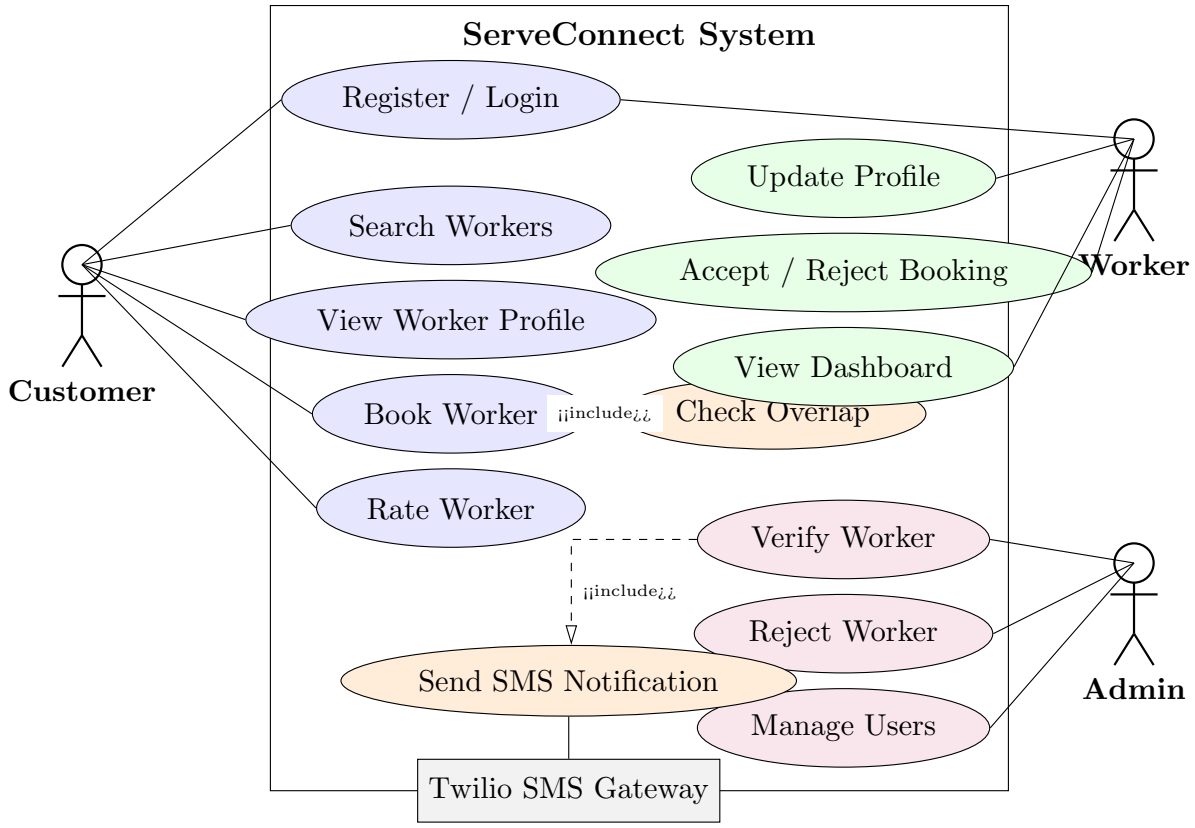


Figure 2.3: Use Case Diagram – ServeConnect

Class Diagram

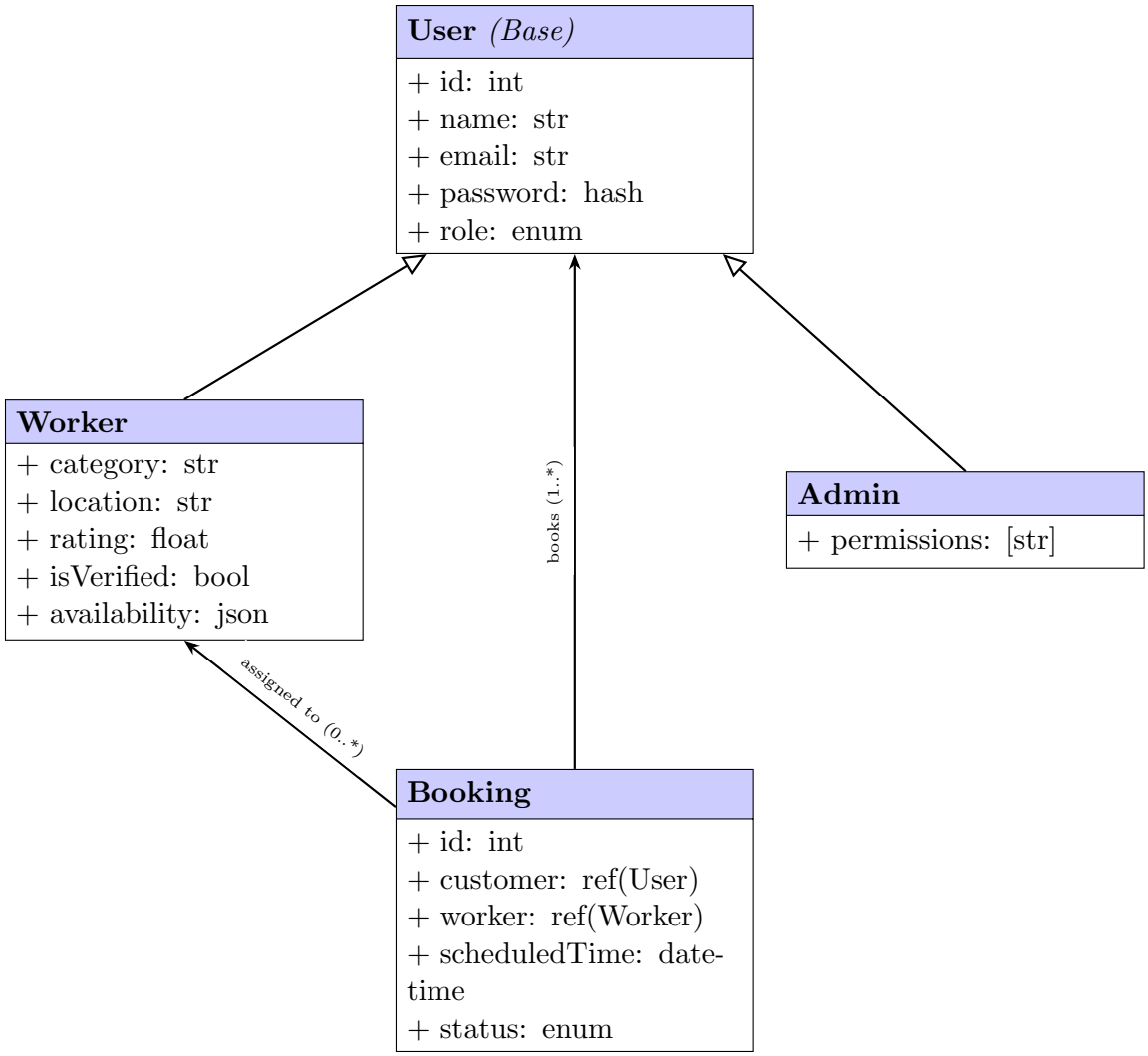


Figure 2.4: Class Diagram – ServeConnect

Sequence Diagram

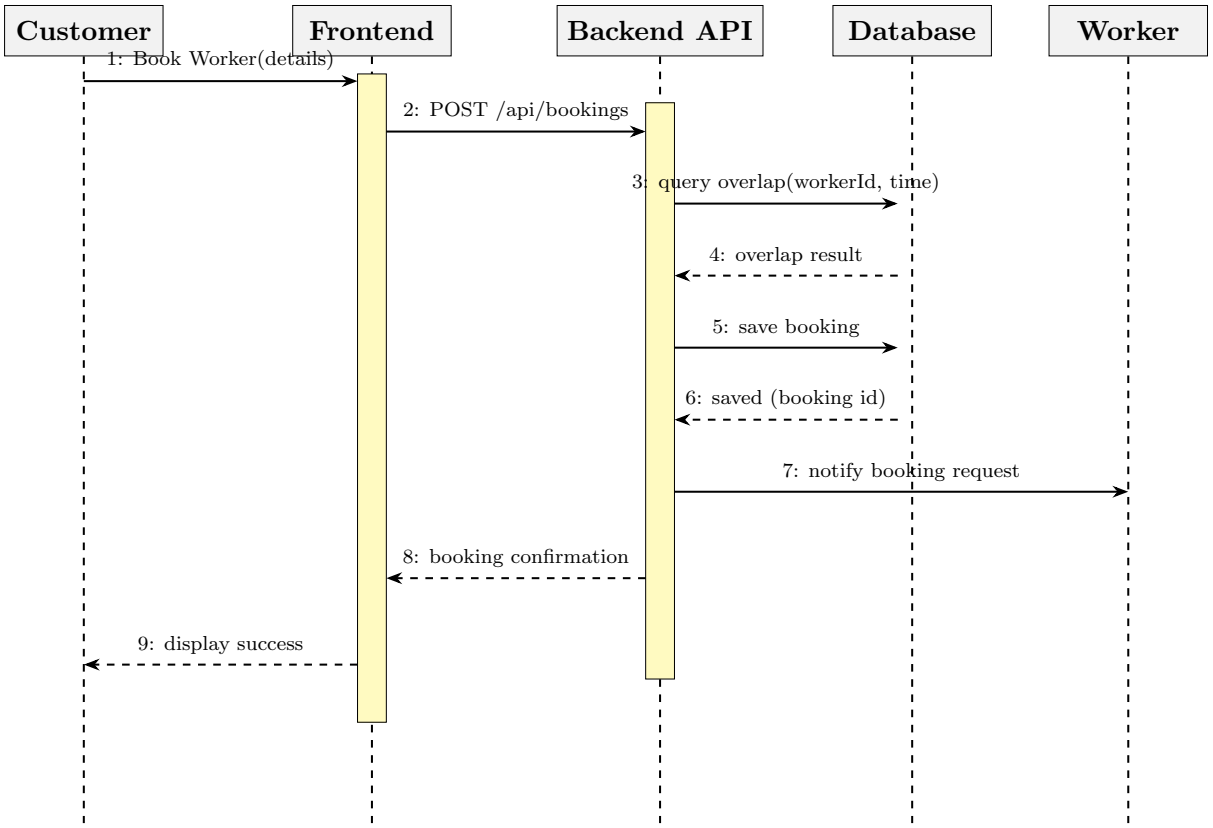


Figure 2.5: Sequence Diagram – Booking Flow

3 Results and Evaluation

3.1 Testing Methodology

- **Unit Testing:** Individual controllers and database schemas (User, Worker, Booking) were tested to ensure proper validation and default values.
- **Integration Testing:** End-to-End sequence flows were evaluated using Jest and Supertest (e.g., `booking.test.js` and `search.test.js`). Tests ensured that the double-booking prevention logic correctly intercepted overlaps and returned a 400 Bad Request before hitting the database.

3.2 Analysis

The prototype successfully met all core objectives for the mid-semester evaluation. The integration of the JWT role-based middleware functions flawlessly to protect routes. The Zomato-style location context executes efficiently on the frontend without interrupting the customer's browsing experience, and the automated Twilio SMS triggers instantaneously upon Admin approval.

4 Conclusion

4.1 Summary of Work

We successfully developed the ServeConnect prototype, establishing a robust Node/Express backend and a modern React frontend, comprising user accounts, verified worker listings, admin dashboards, and an integrated booking system.

4.2 Key Findings

- **Finding 1:** Implementing a centralized React Context for location management significantly improved frontend performance by reducing redundant API calls.
- **Finding 2:** Automating the Admin approval flow with external SMS APIs provides immediate, transparent value and trust to newly onboarded workers.
- **Finding 3:** Enforcing double-booking constraints strictly on the backend database level prevents race conditions when multiple customers try to book the same popular worker.

4.3 Future Work and Recommendations

1. Implementation of Real-Time WebSockets (`Socket.io`) for live chat between customers and workers (Scheduled for Sprint 7).
2. Integration of the Stripe Payment Gateway for secure checkout post-service completion (Scheduled for Sprint 8).
3. Implementation of a Cloudinary-backed Review and Portfolio image upload system to enhance worker profiles.

4.4 Final Remarks

The current working prototype demonstrates a comprehensive application of software engineering principles, successfully bridging modern web development (MERN stack) with real-world service-marketplace requirements such as trust verification and conflict-free scheduling.